FILE: /app/backend/server.py

```
import json
import logging
import os
import re
import uuid
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List, Literal, Optional

from dotenv import load_dotenv
from emergentintegrations.llm.chat import LlmChat, UserMessage
from fastapi import APIRouter, FastAPI, HTTPException
from motor.motor_asyncio import AsyncIOMotorClient
from pydantic import BaseModel, ConfigDict, Field
from starlette.middleware.cors import CORSMiddleware


ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / ".env")

mongo_url = os.environ["MONGO_URL"]
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ["DB_NAME"]]

app = FastAPI()
api_router = APIRouter(prefix="/api")


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()


def parse_iso_date(date_value: Optional[str]) -> Optional[datetime]:
    if not date_value:
        return None
    try:
        return datetime.fromisoformat(date_value.replace("Z", "+00:00"))
    except ValueError:
        return None


def strip_json_fence(text: str) -> str:
    matches = re.findall(r"```(?:json)?\s*(.*?)```", text, flags=re.DOTALL)
    if matches:
        return matches[0].strip()
    return text.strip()


class StatusCheck(BaseModel):
    model_config = ConfigDict(extra="ignore")

    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    client_name: str
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))


class StatusCheckCreate(BaseModel):
    client_name: str


class TaskBase(BaseModel):
    title: str
    subject: str = "General"
    priority: Literal["low", "medium", "high"] = "medium"
    due_date: Optional[str] = None
    estimated_minutes: int = 60
    completed: bool = False


class TaskCreate(TaskBase):
    pass


class TaskUpdate(BaseModel):
    title: Optional[str] = None
    subject: Optional[str] = None
    priority: Optional[Literal["low", "medium", "high"]] = None
    due_date: Optional[str] = None
    estimated_minutes: Optional[int] = None
    completed: Optional[bool] = None


class Task(TaskBase):
```

```
    id: str
    created_at: str
    updated_at: str


class DeadlineBase(BaseModel):
    title: str
    subject: str = "General"
    due_date: str
    reminder_date: Optional[str] = None
    notes: Optional[str] = ""
    completed: bool = False


class DeadlineCreate(DeadlineBase):
    pass


class DeadlineUpdate(BaseModel):
    title: Optional[str] = None
    subject: Optional[str] = None
    due_date: Optional[str] = None
    reminder_date: Optional[str] = None
    notes: Optional[str] = None
    completed: Optional[bool] = None


class Deadline(DeadlineBase):
    id: str
    created_at: str
    updated_at: str


class FocusSessionCreate(BaseModel):
    duration_minutes: int
    started_at: Optional[str] = None
    completed: bool = True


class FocusSession(BaseModel):
    id: str
    duration_minutes: int
    started_at: str
    completed: bool
    created_at: str


class StudyPlanRequest(BaseModel):
    goals: str
    available_hours_per_day: float
    study_days: int = 7
    priorities: Optional[str] = ""


class StudyDay(BaseModel):
    day: str
    topics: List[str]
    focus_minutes: int
    notes: str


class StudyPlanResponse(BaseModel):
    id: str
    goals: str
    priorities: str
    available_hours_per_day: float
    study_days: int
    generated_schedule: List[StudyDay]
    summary: str
    raw_text: str
    created_at: str


class ReminderItem(BaseModel):
    id: str
    title: str
    due_date: str
    days_left: int
    urgency: Literal["overdue", "due_soon", "upcoming"]


class SubjectStat(BaseModel):
    subject: str
    total_tasks: int
```

```
    completed_tasks: int


class AnalyticsSummary(BaseModel):
    tasks_total: int
    tasks_completed: int
    deadlines_upcoming: int
    focus_minutes_total: int
    focus_sessions_count: int
    productivity_score: int
    subject_breakdown: List[SubjectStat] = Field(default_factory=list)


def fallback_schedule(payload: StudyPlanRequest) -> tuple[List[Dict], str]:
    topics = [part.strip() for part in re.split(r",|;", payload.goals) if part.strip()]
    if not topics:
        topics = ["General Revision"]

    plan: List[Dict] = []
    for day_index in range(payload.study_days):
        topic = topics[day_index % len(topics)]
        plan.append(
            {
                "day": f"Day {day_index + 1}",
                "topics": [topic, "Active recall"],
                "focus_minutes": int(payload.available_hours_per_day * 60),
                "notes": "Study in two focused blocks with a short revision at the end.",
            }
        )

    return plan, "Structured fallback plan generated."


def extract_schedule_source_and_summary(loaded: object) -> tuple[List[Dict], str]:
    if isinstance(loaded, dict):
        summary = loaded["summary"] if isinstance(loaded.get("summary"), str) else "AI generated plan created."
        schedule = loaded["schedule"] if isinstance(loaded.get("schedule"), list) else []
        return schedule, summary

    if isinstance(loaded, list):
        return loaded, "AI generated plan created."

    return [], "AI generated plan created."


def normalize_topics(topics: object) -> List[str]:
    if isinstance(topics, str):
        return [topics]
    if isinstance(topics, list) and topics:
        return [str(topic) for topic in topics]
    return ["General topic"]


def normalize_schedule_entries(schedule_source: List[Dict], payload: StudyPlanRequest) -> List[Dict]:
    normalized_schedule: List[Dict] = []
    default_focus_minutes = int(payload.available_hours_per_day * 60)

    for idx, item in enumerate(schedule_source[: payload.study_days]):
        if not isinstance(item, dict):
            continue

        normalized_schedule.append(
            {
                "day": str(item.get("day", f"Day {idx + 1}")),
                "topics": normalize_topics(item.get("topics")),
                "focus_minutes": int(item.get("focus_minutes", default_focus_minutes)),
                "notes": str(item.get("notes", "Review and recap for 10 minutes.")),
            }
        )

    return normalized_schedule


def pad_schedule_days(normalized_schedule: List[Dict], payload: StudyPlanRequest) -> List[Dict]:
    while len(normalized_schedule) < payload.study_days:
        next_day = len(normalized_schedule) + 1
        normalized_schedule.append(
            {
                "day": f"Day {next_day}",
                "topics": ["Revision Block"],
                "focus_minutes": int(payload.available_hours_per_day * 60),
                "notes": "Use this day for weak-topic reinforcement.",
            }
        )
```

```
252|     return normalized_schedule
253|
254|
255| def parse_plan_response(raw_text: str, payload: StudyPlanRequest) -> tuple[List[Dict], str]:
256|     cleaned = strip_json_fence(raw_text)
257|     try:
258|         loaded = json.loads(cleaned)
259|     except json.JSONDecodeError:
260|         return fallback_schedule(payload)
261|
262|     schedule_source, summary = extract_schedule_source_and_summary(loaded)
263|     normalized_schedule = normalize_schedule_entries(schedule_source, payload)
264|
265|     if not normalized_schedule:
266|         return fallback_schedule(payload)
267|
268|     return pad_schedule_days(normalized_schedule, payload), summary
269|
270|
271| def count_upcoming_deadlines(deadlines: List[Dict], now: datetime) -> int:
272|     return sum(
273|         1
274|         for deadline in deadlines
275|         if (due_date := parse_iso_date(deadline.get("due_date"))) and due_date >= now
276|     )
277|
278|
279| def build_subject_breakdown(tasks: List[Dict]) -> List[SubjectStat]:
280|     subject_map: Dict[str, Dict[str, int]] = {}
281|     for task in tasks:
282|         subject = task.get("subject", "General")
283|         if subject not in subject_map:
284|             subject_map[subject] = {"total": 0, "completed": 0}
285|         subject_map[subject]["total"] += 1
286|         if task.get("completed"):
287|             subject_map[subject]["completed"] += 1
288|
289|     subject_breakdown = [
290|         SubjectStat(subject=subject, total_tasks=stats["total"], completed_tasks=stats["completed"])
291|         for subject, stats in subject_map.items()
292|     ]
293|     subject_breakdown.sort(key=lambda item: item.total_tasks, reverse=True)
294|     return subject_breakdown
295|
296|
297| def calculate_productivity_score(tasks_total: int, tasks_completed: int, focus_minutes_total: int) -> int:
298|     completion_ratio = tasks_completed / tasks_total if tasks_total else 0
299|     focus_component = min(50, focus_minutes_total // 10)
300|     return min(100, int(completion_ratio * 50 + focus_component))
301|
302|
303| @api_router.get("/")
304| async def root():
305|     return {"message": "AI Study Planner API is running"}
306|
307|
308| @api_router.post("/status", response_model=StatusCheck)
309| async def create_status_check(input: StatusCheckCreate):
310|     status_obj = StatusCheck(**input.model_dump())
311|     doc = status_obj.model_dump()
312|     doc["timestamp"] = doc["timestamp"].isoformat()
313|     await db.status_checks.insert_one(doc)
314|     return status_obj
315|
316|
317| @api_router.get("/status", response_model=List[StatusCheck])
318| async def get_status_checks():
319|     status_checks = await db.status_checks.find({}, {"_id": 0}).to_list(1000)
320|     for check in status_checks:
321|         if isinstance(check.get("timestamp"), str):
322|             check["timestamp"] = datetime.fromisoformat(check["timestamp"])
323|     return status_checks
324|
325|
326| @api_router.post("/plans/generate", response_model=StudyPlanResponse)
327| async def generate_study_plan(payload: StudyPlanRequest):
328|     llm_key = os.environ.get("EMERGENT_LLM_KEY")
329|     if not llm_key:
330|         raise HTTPException(status_code=500, detail="Missing EMERGENT_LLM_KEY in backend environment")
331|
332|     prompt = (
333|         "Create a realistic student study plan. Return only JSON with keys: "
334|         "summary and schedule. schedule must be an array of objects with day, topics, focus_minutes, notes. "
335|         f"Goals: {payload.goals}. Priorities: {payload.priorities}. "
```

```
336|         f"Hours per day: {payload.available_hours_per_day}. Number of days: {payload.study_days}."
337|     )
338|
339|     chat = LlmChat(
340|         api_key=llm_key,
341|         session_id=f"study-plan-{uuid.uuid4()}",
342|         system_message="You are a study productivity assistant. Output strict JSON only.",
343|     ).with_model("openai", "gpt-5.2")
344|
345|     ai_response = await chat.send_message(UserMessage(text=prompt))
346|     raw_text = ai_response if isinstance(ai_response, str) else str(ai_response)
347|
348|     generated_schedule, summary = parse_plan_response(raw_text, payload)
349|
350|     plan_doc = {
351|         "id": str(uuid.uuid4()),
352|         "goals": payload.goals,
353|         "priorities": payload.priorities or "",
354|         "available_hours_per_day": payload.available_hours_per_day,
355|         "study_days": payload.study_days,
356|         "generated_schedule": generated_schedule,
357|         "summary": summary,
358|         "raw_text": raw_text,
359|         "created_at": now_iso(),
360|     }
361|
362|     await db.study_plans.insert_one(plan_doc.copy())
363|     return StudyPlanResponse(**plan_doc)
364|
365|
366| @api_router.get("/plans", response_model=List[StudyPlanResponse])
367| async def get_study_plans():
368|     plans = await db.study_plans.find({}, {"_id": 0}).sort("created_at", -1).to_list(50)
369|     return plans
370|
371|
372| @api_router.post("/tasks", response_model=Task)
373| async def create_task(payload: TaskCreate):
374|     timestamp = now_iso()
375|     task_doc = payload.model_dump()
376|     task_doc.update({"id": str(uuid.uuid4()), "created_at": timestamp, "updated_at": timestamp})
377|     await db.tasks.insert_one(task_doc.copy())
378|     return Task(**task_doc)
379|
380|
381| @api_router.get("/tasks", response_model=List[Task])
382| async def get_tasks():
383|     tasks = await db.tasks.find({}, {"_id": 0}).sort("created_at", -1).to_list(500)
384|     return tasks
385|
386|
387| @api_router.put("/tasks/{task_id}", response_model=Task)
388| async def update_task(task_id: str, payload: TaskUpdate):
389|     updates = {key: value for key, value in payload.model_dump().items() if value is not None}
390|     updates["updated_at"] = now_iso()
391|     result = await db.tasks.update_one({"id": task_id}, {"$set": updates})
392|     if result.matched_count == 0:
393|         raise HTTPException(status_code=404, detail="Task not found")
394|
395|     updated_task = await db.tasks.find_one({"id": task_id}, {"_id": 0})
396|     if not updated_task:
397|         raise HTTPException(status_code=404, detail="Task not found")
398|     return updated_task
399|
400|
401| @api_router.delete("/tasks/{task_id}")
402| async def delete_task(task_id: str):
403|     result = await db.tasks.delete_one({"id": task_id})
404|     if result.deleted_count == 0:
405|         raise HTTPException(status_code=404, detail="Task not found")
406|     return {"success": True, "message": "Task deleted"}
407|
408|
409| @api_router.post("/deadlines", response_model=Deadline)
410| async def create_deadline(payload: DeadlineCreate):
411|     timestamp = now_iso()
412|     deadline_doc = payload.model_dump()
413|     deadline_doc.update({"id": str(uuid.uuid4()), "created_at": timestamp, "updated_at": timestamp})
414|     await db.deadlines.insert_one(deadline_doc.copy())
415|     return Deadline(**deadline_doc)
416|
417|
418| @api_router.get("/deadlines", response_model=List[Deadline])
419| async def get_deadlines():
```

```
    deadlines = await db.deadlines.find({}, {"_id": 0}).sort("due_date", 1).to_list(500)
    return deadlines


@api_router.put("/deadlines/{deadline_id}", response_model=Deadline)
async def update_deadline(deadline_id: str, payload: DeadlineUpdate):
    updates = {key: value for key, value in payload.model_dump().items() if value is not None}
    updates["updated_at"] = now_iso()
    result = await db.deadlines.update_one({"id": deadline_id}, {"$set": updates})
    if result.matched_count == 0:
        raise HTTPException(status_code=404, detail="Deadline not found")

    updated_deadline = await db.deadlines.find_one({"id": deadline_id}, {"_id": 0})
    if not updated_deadline:
        raise HTTPException(status_code=404, detail="Deadline not found")
    return updated_deadline


@api_router.delete("/deadlines/{deadline_id}")
async def delete_deadline(deadline_id: str):
    result = await db.deadlines.delete_one({"id": deadline_id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Deadline not found")
    return {"success": True, "message": "Deadline deleted"}


@api_router.get("/reminders", response_model=List[ReminderItem])
async def get_deadline_reminders():
    now = datetime.now(timezone.utc)
    deadlines = await db.deadlines.find({"completed": False}, {"_id": 0}).to_list(500)
    reminders: List[ReminderItem] = []

    for item in deadlines:
        due_date = parse_iso_date(item.get("due_date"))
        if not due_date:
            continue
        days_left = (due_date.date() - now.date()).days
        urgency: Literal["overdue", "due_soon", "upcoming"]
        if days_left < 0:
            urgency = "overdue"
        elif days_left <= 3:
            urgency = "due_soon"
        else:
            urgency = "upcoming"

        if urgency in {"overdue", "due_soon"}:
            reminders.append(
                ReminderItem(
                    id=item["id"],
                    title=item["title"],
                    due_date=item["due_date"],
                    days_left=days_left,
                    urgency=urgency,
                )
            )

    reminders.sort(key=lambda reminder: reminder.days_left)
    return reminders


@api_router.post("/focus-sessions", response_model=FocusSession)
async def create_focus_session(payload: FocusSessionCreate):
    timestamp = now_iso()
    session_doc = {
        "id": str(uuid.uuid4()),
        "duration_minutes": payload.duration_minutes,
        "started_at": payload.started_at or timestamp,
        "completed": payload.completed,
        "created_at": timestamp,
    }
    await db.focus_sessions.insert_one(session_doc.copy())
    return FocusSession(**session_doc)


@api_router.get("/focus-sessions", response_model=List[FocusSession])
async def get_focus_sessions():
    sessions = await db.focus_sessions.find({}, {"_id": 0}).sort("created_at", -1).to_list(300)
    return sessions


@api_router.get("/analytics/summary", response_model=AnalyticsSummary)
async def get_analytics_summary():
    now = datetime.now(timezone.utc)

```

```
    tasks = await db.tasks.find({}, {"_id": 0}).to_list(2000)
    deadlines = await db.deadlines.find({"completed": False}, {"_id": 0}).to_list(2000)
    focus_sessions = await db.focus_sessions.find({}, {"_id": 0}).to_list(2000)

    tasks_total = len(tasks)
    tasks_completed = len([task for task in tasks if task.get("completed")])
    focus_minutes_total = sum(int(session.get("duration_minutes", 0)) for session in focus_sessions)

    upcoming_deadlines = count_upcoming_deadlines(deadlines, now)
    subject_breakdown = build_subject_breakdown(tasks)
    productivity_score = calculate_productivity_score(tasks_total, tasks_completed, focus_minutes_total)

    return AnalyticsSummary(
        tasks_total=tasks_total,
        tasks_completed=tasks_completed,
        deadlines_upcoming=upcoming_deadlines,
        focus_minutes_total=focus_minutes_total,
        focus_sessions_count=len(focus_sessions),
        productivity_score=productivity_score,
        subject_breakdown=subject_breakdown,
    )


app.include_router(api_router)

app.add_middleware(
    CORSMiddleware,
    allow_credentials=True,
    allow_origins=os.environ.get("CORS_ORIGINS", "*").split(","),
    allow_methods=["*"],
    allow_headers=["*"],
)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
)
logger = logging.getLogger(__name__)


@app.on_event("shutdown")
async def shutdown_db_client():
    client.close()
```

FILE: /app/backend/requirements.txt

```
fastapi==0.110.1
uvicorn==0.25.0
boto3>=1.34.129
requests-oauthlib>=2.0.0
cryptography>=42.0.8
python-dotenv>=1.0.1
pymongo==4.5.0
pydantic>=2.6.4
email-validator>=2.2.0
pyjwt>=2.10.1
bcrypt==4.1.3
passlib>=1.7.4
tzdata>=2024.2
motor==3.3.1
pytest>=8.0.0
black>=24.1.1
isort>=5.13.2
flake8>=7.0.0
mypy>=1.8.0
python-jose>=3.3.0
requests>=2.31.0
pandas>=2.2.0
numpy>=1.26.0
python-multipart>=0.0.9
jq>=1.6.0
typer>=0.9.0
emergentintegrations==0.1.0
```

FILE: /app/frontend/src/App.js

```
import "@/App.css";
import { useEffect } from "react";
import { BrowserRouter, Navigate, Route, Routes } from "react-router-dom";
import { ThemeProvider } from "next-themes";
import { Toaster } from "@/components/ui/sonner";
import { TopNav } from "@/components/TopNav";
import { useStudyData } from "@/hooks/useStudyData";
import DashboardPage from "@/pages/DashboardPage";
import PlannerPage from "@/pages/PlannerPage";
import TasksPage from "@/pages/TasksPage";
```

```
import FocusPage from "@/pages/FocusPage";
import DeadlinesPage from "@/pages/DeadlinesPage";
import AnalyticsPage from "@/pages/AnalyticsPage";

const PlannerRoutes = ({
  generatingPlan,
  submittingTask,
  submittingDeadline,
  tasks,
  deadlines,
  reminders,
  plans,
  focusSessions,
  analytics,
  addTask,
  toggleTask,
  deleteTask,
  updateTaskDetails,
  addDeadline,
  toggleDeadline,
  deleteDeadline,
  updateDeadlineDetails,
  logFocusSession,
  generatePlan,
}) => {
  return (
    <Routes>
      <Route
        path="/dashboard"
        element={<DashboardPage tasks={tasks} deadlines={deadlines} analytics={analytics} plans={plans} />}
      />
      <Route
        path="/planner"
        element={<PlannerPage generatingPlan={generatingPlan} latestPlan={plans[0]} onGeneratePlan={generatePlan} />}
      />
      <Route
        path="/tasks"
        element={
          <TasksPage
            submittingTask={submittingTask}
            tasks={tasks}
            onAddTask={addTask}
            onToggleTask={toggleTask}
            onDeleteTask={deleteTask}
            onUpdateTask={updateTaskDetails}
          />
        }
      />
      <Route path="/focus" element={<FocusPage focusSessions={focusSessions} onLogSession={logFocusSession} />} />
      <Route
        path="/deadlines"
        element={
          <DeadlinesPage
            submittingDeadline={submittingDeadline}
            deadlines={deadlines}
            reminders={reminders}
            onAddDeadline={addDeadline}
            onToggleDeadline={toggleDeadline}
            onDeleteDeadline={deleteDeadline}
            onUpdateDeadline={updateDeadlineDetails}
          />
        }
      />
      <Route path="/analytics" element={<AnalyticsPage analytics={analytics} />} />
      <Route path="*" element={<Navigate to="/dashboard" replace />} />
    </Routes>
  );
};

function App() {
  const {
    loading,
    generatingPlan,
    submittingTask,
    submittingDeadline,
    tasks,
    deadlines,
    reminders,
    plans,
    focusSessions,
    analytics,
    addTask,
    toggleTask,
    deleteTask,
```

```
    updateTaskDetails,
    addDeadline,
    toggleDeadline,
    deleteDeadline,
    updateDeadlineDetails,
    logFocusSession,
    generatePlan,
  } = useStudyData();

  useEffect(() => {
    const hideInjectedBadge = () => {
      const candidates = document.querySelectorAll("a, div, span");
      candidates.forEach((node) => {
        if ((node.textContent || "").includes("Made with Emergent")) {
          node.style.setProperty("display", "none", "important");
          node.style.setProperty("visibility", "hidden", "important");
          node.style.setProperty("opacity", "0", "important");
          node.style.setProperty("pointer-events", "none", "important");
        }
      });
    };

    hideInjectedBadge();
    const observer = new MutationObserver(() => hideInjectedBadge());
    observer.observe(document.body, { childList: true, subtree: true });

    return () => observer.disconnect();
  }, []);

  return (
    <ThemeProvider attribute="class" defaultTheme="light" enableSystem={false}>
      <BrowserRouter>
        <div className="planner-bg" data-testid="study-planner-app-shell">
          <TopNav />
          <main className="page-shell">
            {loading ? (
              <div
                className="brutal-card bg-white p-8 text-base md:text-lg"
                data-testid="global-loading-card"
              >
                Loading your study space...
              </div>
            ) : (
              <PlannerRoutes
                generatingPlan={generatingPlan}
                submittingTask={submittingTask}
                submittingDeadline={submittingDeadline}
                tasks={tasks}
                deadlines={deadlines}
                reminders={reminders}
                plans={plans}
                focusSessions={focusSessions}
                analytics={analytics}
                addTask={addTask}
                toggleTask={toggleTask}
                deleteTask={deleteTask}
                updateTaskDetails={updateTaskDetails}
                addDeadline={addDeadline}
                toggleDeadline={toggleDeadline}
                deleteDeadline={deleteDeadline}
                updateDeadlineDetails={updateDeadlineDetails}
                logFocusSession={logFocusSession}
                generatePlan={generatePlan}
              />
            )}
          </main>
          <Toaster data-testid="toast-root" position="top-right" richColors closeButton />
        </div>
      </BrowserRouter>
    </ThemeProvider>
  );
}

export default App;
```

FILE: /app/frontend/src/App.css

```
.planner-bg {
  min-height: 100vh;
  background-color: #fafaf9;
  background-image:
    radial-gradient(circle at 20% 10%, rgba(191, 219, 254, 0.5), transparent 35%),
    radial-gradient(circle at 80% 90%, rgba(187, 247, 208, 0.55), transparent 35%),
    radial-gradient(circle at 60% 40%, rgba(233, 213, 255, 0.45), transparent 40%);
}
```

```

.page-shell {
  padding: 2rem 1rem 7rem;
}

@media (min-width: 768px) {
  .page-shell {
    padding: 2.5rem 2rem 8rem;
  }
}

.brutal-card {
  border: 2px solid #18181b;
  border-radius: 0;
  box-shadow: 4px 4px 0 0 #18181b;
  transition: transform 180ms ease, box-shadow 180ms ease;
}

.brutal-card:hover {
  transform: translate(2px, 2px);
  box-shadow: 0 0 0 0 #18181b;
}

.brutal-input {
  border: 0;
  border-bottom: 2px solid #18181b;
  border-radius: 0;
  padding: 0.5rem 0;
  background: transparent;
  transition: border-width 120ms ease;
}

.brutal-input:focus {
  outline: none;
  border-bottom-width: 4px;
}

.timer-font {
  font-family: "JetBrains Mono", monospace;
}

.card-appear {
  animation: cardFadeUp 350ms ease forwards;
}

@keyframes cardFadeUp {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

FILE: /app/frontend/src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "@fontsource/syne/700.css";
import "@fontsource/outfit/400.css";
import "@fontsource/outfit/500.css";
import "@fontsource/jetbrains-mono/500.css";
import "@/index.css";
import App from "@/App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

FILE: /app/frontend/src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

```

```
11| code {
12|   font-family: "JetBrains Mono", monospace;
13| }
14|
15| @layer base {
16|   :root {
17|     --background: 48 33% 98%;
18|     --foreground: 240 10% 10%;
19|     --card: 0 0% 100%;
20|     --card-foreground: 240 10% 10%;
21|     --popover: 0 0% 100%;
22|     --popover-foreground: 240 10% 10%;
23|     --primary: 240 10% 10%;
24|     --primary-foreground: 0 0% 98%;
25|     --secondary: 240 17% 96%;
26|     --secondary-foreground: 240 10% 10%;
27|     --muted: 220 14% 96%;
28|     --muted-foreground: 240 5% 40%;
29|     --accent: 213 88% 87%;
30|     --accent-foreground: 240 10% 10%;
31|     --destructive: 0 76% 62%;
32|     --destructive-foreground: 0 0% 98%;
33|     --border: 240 10% 10%;
34|     --input: 240 10% 10%;
35|     --ring: 240 10% 10%;
36|     --chart-1: 6 92% 81%;
37|     --chart-2: 210 88% 87%;
38|     --chart-3: 136 71% 84%;
39|     --chart-4: 31 90% 84%;
40|     --chart-5: 273 100% 92%;
41|     --radius: 0rem;
42|   }
43|
44|   .dark {
45|     --background: 240 12% 7%;
46|     --foreground: 0 0% 96%;
47|     --card: 240 10% 10%;
48|     --card-foreground: 0 0% 96%;
49|     --popover: 240 10% 10%;
50|     --popover-foreground: 0 0% 96%;
51|     --primary: 0 0% 96%;
52|     --primary-foreground: 240 10% 10%;
53|     --secondary: 240 8% 20%;
54|     --secondary-foreground: 0 0% 96%;
55|     --muted: 240 8% 20%;
56|     --muted-foreground: 240 5% 70%;
57|     --accent: 240 8% 20%;
58|     --accent-foreground: 0 0% 96%;
59|     --destructive: 0 62% 30%;
60|     --destructive-foreground: 0 0% 96%;
61|     --border: 0 0% 96%;
62|     --input: 240 8% 20%;
63|     --ring: 0 0% 96%;
64|   }
65| }
66|
67| @layer base {
68|   * {
69|     @apply border-border;
70|   }
71|
72|   body {
73|     @apply bg-background text-foreground;
74|     font-family: "Outfit", sans-serif;
75|   }
76|
77|   h1,
78|   h2,
79|   h3,
80|   h4 {
81|     font-family: "Syne", sans-serif;
82|     letter-spacing: -0.02em;
83|   }
84| }
85|
86| @layer base {
87|   [data-debug-wrapper="true"] {
88|     display: contents !important;
89|   }
90|
91|   [data-debug-wrapper="true"] > * {
92|     margin-left: inherit;
93|     margin-right: inherit;
94|     margin-top: inherit;
```

```
    margin-bottom: inherit;
    padding-left: inherit;
    padding-right: inherit;
    padding-top: inherit;
    padding-bottom: inherit;
    column-gap: inherit;
    row-gap: inherit;
    gap: inherit;
    border-left-width: inherit;
    border-right-width: inherit;
    border-top-width: inherit;
    border-bottom-width: inherit;
    border-left-style: inherit;
    border-right-style: inherit;
    border-top-style: inherit;
    border-bottom-style: inherit;
    border-left-color: inherit;
    border-right-color: inherit;
    border-top-color: inherit;
    border-bottom-color: inherit;
  }
}

#emergent-badge {
  display: none !important;
  visibility: hidden !important;
  opacity: 0 !important;
  pointer-events: none !important;
}
```

FILE: /app/frontend/src/api/studyApi.js

```
import axios from "axios";

const API_BASE = `${process.env.REACT_APP_BACKEND_URL}/api`;

const api = axios.create({
  baseURL: API_BASE,
  headers: {
    "Content-Type": "application/json",
  },
});

export const getTasks = async () => (await api.get("/tasks")).data;
export const createTask = async (payload) => (await api.post("/tasks", payload)).data;
export const updateTask = async (taskId, payload) => (await api.put(`/tasks/${taskId}`, payload)).data;
export const removeTask = async (taskId) => (await api.delete(`/tasks/${taskId}`)).data;

export const getDeadlines = async () => (await api.get("/deadlines")).data;
export const createDeadline = async (payload) => (await api.post("/deadlines", payload)).data;
export const updateDeadline = async (deadlineId, payload) => (await api.put(`/deadlines/${deadlineId}`, payload)).data;
export const removeDeadline = async (deadlineId) => (await api.delete(`/deadlines/${deadlineId}`)).data;
export const getReminders = async () => (await api.get("/reminders")).data;

export const getPlans = async () => (await api.get("/plans")).data;
export const generateStudyPlan = async (payload) => (await api.post("/plans/generate", payload)).data;

export const getFocusSessions = async () => (await api.get("/focus-sessions")).data;
export const createFocusSession = async (payload) => (await api.post("/focus-sessions", payload)).data;

export const getAnalyticsSummary = async () => (await api.get("/analytics/summary")).data;
```

FILE: /app/frontend/src/hooks/useStudyData.js

```
import { useCallback, useEffect, useState } from "react";
import { toast } from "@/components/ui/sonner";
import {
  createDeadline,
  createFocusSession,
  createTask,
  generateStudyPlan,
  getAnalyticsSummary,
  getDeadlines,
  getFocusSessions,
  getPlans,
  getReminders,
  getTasks,
  removeDeadline,
  removeTask,
  updateDeadline,
  updateTask,
} from "@/api/studyApi";

const EMPTY_ANALYTICS = {
  tasks_total: 0,
  tasks_completed: 0,
```

```
  deadlines_upcoming: 0,
  focus_minutes_total: 0,
  focus_sessions_count: 0,
  productivity_score: 0,
  subject_breakdown: [],
};

export const useStudyData = () => {
  const [loading, setLoading] = useState(true);
  const [generatingPlan, setGeneratingPlan] = useState(false);
  const [submittingTask, setSubmittingTask] = useState(false);
  const [submittingDeadline, setSubmittingDeadline] = useState(false);
  const [tasks, setTasks] = useState([]);
  const [deadlines, setDeadlines] = useState([]);
  const [reminders, setReminders] = useState([]);
  const [plans, setPlans] = useState([]);
  const [focusSessions, setFocusSessions] = useState([]);
  const [analytics, setAnalytics] = useState(EMPTY_ANALYTICS);

  const loadAllData = useCallback(async () => {
    try {
      const [taskData, deadlineData, reminderData, planData, sessionData, analyticsData] = await Promise.all([
        getTasks(),
        getDeadlines(),
        getReminders(),
        getPlans(),
        getFocusSessions(),
        getAnalyticsSummary(),
      ]);
      setTasks(taskData);
      setDeadlines(deadlineData);
      setReminders(reminderData);
      setPlans(planData);
      setFocusSessions(sessionData);
      setAnalytics(analyticsData);
    } catch {
      toast.error("Unable to load planner data. Please refresh.");
    } finally {
      setLoading(false);
    }
  }, []);

  const mutateAndRefresh = async ({ action, successMessage, errorMessage, onStart, onDone }) => {
    try {
      if (onStart) {
        onStart();
      }
      await action();
      toast.success(successMessage);
      await loadAllData();
    } catch {
      toast.error(errorMessage);
    } finally {
      if (onDone) {
        onDone();
      }
    }
  };

  useEffect(() => {
    void loadAllData();
  }, [loadAllData]);

  const addTask = async (payload) => {
    await mutateAndRefresh({
      action: () => createTask(payload),
      successMessage: "Task added.",
      errorMessage: "Could not add task.",
      onStart: () => setSubmittingTask(true),
      onDone: () => setSubmittingTask(false),
    });
  };

  const toggleTask = async (task) => {
    await mutateAndRefresh({
      action: () => updateTask(task.id, { completed: !task.completed }),
      successMessage: task.completed ? "Task reopened." : "Task completed.",
      errorMessage: "Could not update task.",
    });
  };

  const deleteTask = async (taskId) => {
    await mutateAndRefresh({
      action: () => removeTask(taskId),
```

```
107|       successMessage: "Task deleted.",
108|       errorMessage: "Could not delete task.",
109|     });
110|   };
111|
112|   const updateTaskDetails = async (taskId, payload) => {
113|     await mutateAndRefresh({
114|       action: () => updateTask(taskId, payload),
115|       successMessage: "Task updated.",
116|       errorMessage: "Could not update task.",
117|     });
118|   };
119|
120|   const addDeadline = async (payload) => {
121|     await mutateAndRefresh({
122|       action: () => createDeadline(payload),
123|       successMessage: "Deadline added.",
124|       errorMessage: "Could not add deadline.",
125|       onStart: () => setSubmittingDeadline(true),
126|       onDone: () => setSubmittingDeadline(false),
127|     });
128|   };
129|
130|   const toggleDeadline = async (deadline) => {
131|     await mutateAndRefresh({
132|       action: () => updateDeadline(deadline.id, { completed: !deadline.completed }),
133|       successMessage: deadline.completed ? "Deadline moved back to active." : "Deadline marked complete.",
134|       errorMessage: "Could not update deadline.",
135|     });
136|   };
137|
138|   const deleteDeadline = async (deadlineId) => {
139|     await mutateAndRefresh({
140|       action: () => removeDeadline(deadlineId),
141|       successMessage: "Deadline removed.",
142|       errorMessage: "Could not delete deadline.",
143|     });
144|   };
145|
146|   const updateDeadlineDetails = async (deadlineId, payload) => {
147|     await mutateAndRefresh({
148|       action: () => updateDeadline(deadlineId, payload),
149|       successMessage: "Deadline updated.",
150|       errorMessage: "Could not update deadline.",
151|     });
152|   };
153|
154|   const logFocusSession = async (durationMinutes) => {
155|     await mutateAndRefresh({
156|       action: () =>
157|         createFocusSession({
158|           duration_minutes: durationMinutes,
159|           completed: true,
160|           started_at: new Date().toISOString(),
161|         }),
162|       successMessage: "Focus session logged.",
163|       errorMessage: "Unable to log focus session.",
164|     });
165|   };
166|
167|   const generatePlan = async (payload) => {
168|     await mutateAndRefresh({
169|       action: () => generateStudyPlan(payload),
170|       successMessage: "AI schedule generated.",
171|       errorMessage: "AI plan generation failed.",
172|       onStart: () => setGeneratingPlan(true),
173|       onDone: () => setGeneratingPlan(false),
174|     });
175|   };
176|
177|   return {
178|     loading,
179|     generatingPlan,
180|     submittingTask,
181|     submittingDeadline,
182|     tasks,
183|     deadlines,
184|     reminders,
185|     plans,
186|     focusSessions,
187|     analytics,
188|     addTask,
189|     toggleTask,
190|     deleteTask,
```

```
 191|     updateTaskDetails,
 192|     addDeadline,
 193|     toggleDeadline,
 194|     deleteDeadline,
 195|     updateDeadlineDetails,
 196|     logFocusSession,
 197|     generatePlan,
 198|   };
 199| };
```

FILE: /app/frontend/src/components/TopNav.jsx

```
   1| import { Link, NavLink } from "react-router-dom";
   2| import { BarChart3, Brain, CalendarClock, CheckSquare, House, Timer } from "lucide-react";
   3|
   4| const routes = [
   5|   { to: "/dashboard", label: "Dashboard", icon: House },
   6|   { to: "/planner", label: "AI Planner", icon: Brain },
   7|   { to: "/tasks", label: "Tasks", icon: CheckSquare },
   8|   { to: "/focus", label: "Focus", icon: Timer },
   9|   { to: "/deadlines", label: "Deadlines", icon: CalendarClock },
  10|   { to: "/analytics", label: "Analytics", icon: BarChart3 },
  11| ];
  12|
  13| export const TopNav = () => {
  14|   return (
  15|     <header className="sticky top-0 z-40 bg-[#fafaf9]/90 px-4 pb-4 pt-6 backdrop-blur md:px-8"
data-testid="top-nav-wrapper">
  16|       <div className="mb-5 flex flex-wrap items-center justify-between gap-4">
  17|         <Link
  18|           to="/dashboard"
  19|           className="flex items-center gap-3 text-[#18181b] transition-colors hover:text-zinc-700"
  20|           data-testid="brand-home-link"
  21|         >
  22|           <span className="inline-flex h-10 w-10 items-center justify-center border-2 border-[#18181b] bg-[#bbf7d0] text-lg
font-bold">
  23|             SP
  24|           </span>
  25|           <div>
  26|             <h1 className="text-2xl sm:text-3xl">Study Planner AI</h1>
  27|             <p className="text-sm text-zinc-600" data-testid="brand-subtitle">
  28|               Plan smarter, focus deeper, finish earlier.
  29|             </p>
  30|           </div>
  31|         </Link>
  32|       </div>
  33|
  34|       <nav className="flex flex-wrap gap-3" data-testid="top-nav-links">
  35|         {routes.map(({ to, label, icon: Icon }) => (
  36|           <NavLink
  37|             key={to}
  38|             to={to}
  39|             className={({ isActive }) =>
  40|               [
  41|                 "inline-flex items-center gap-2 rounded-full border-2 border-[#18181b] px-4 py-2 text-sm font-medium
transition-transform",
  42|                 isActive ? "bg-[#18181b] text-white" : "bg-white text-[#18181b] hover:-translate-y-1",
  43|               ].join(" ")
  44|             }
  45|             data-testid={`nav-link-${label.toLowerCase().replace(/\s+/g, "-")}`}
  46|           >
  47|             <Icon size={16} strokeWidth={2} />
  48|             {label}
  49|           </NavLink>
  50|         ))}
  51|       </nav>
  52|     </header>
  53|   );
  54| };
```

FILE: /app/frontend/src/components/BrutalCard.jsx

```
   1| import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
   2|
   3| export const BrutalCard = ({ title, subtitle, className = "", cardTestId, children }) => {
   4|   return (
   5|     <Card className={`brutal-card bg-white ${className}`} data-testid={cardTestId}>
   6|       {(title || subtitle) && (
   7|         <CardHeader className="space-y-2 p-6">
   8|           {title ? (
   9|             <CardTitle className="text-xl text-[#18181b]" data-testid={`${cardTestId}-title`}>
  10|               {title}
  11|             </CardTitle>
  12|           ) : null}
  13|           {subtitle ? (
  14|             <p className="text-sm text-zinc-700" data-testid={`${cardTestId}-subtitle`}>
```

```
  15|              {subtitle}
  16|            </p>
  17|          ) : null}
  18|        </CardHeader>
  19|      )}
  20|      <CardContent className="p-6 pt-0">{children}</CardContent>
  21|    </Card>
  22|  );
  23| };
```

FILE: /app/frontend/src/components/DatePickerField.jsx

```
   1| import { CalendarIcon } from "lucide-react";
   2| import { Button } from "@/components/ui/button";
   3| import { Calendar } from "@/components/ui/calendar";
   4| import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";
   5|
   6| const toDateObject = (value) => {
   7|   if (!value) {
   8|     return undefined;
   9|   }
  10|   const parsed = new Date(value);
  11|   return Number.isNaN(parsed.getTime()) ? undefined : parsed;
  12| };
  13|
  14| const toDisplayValue = (value) => {
  15|   const dateValue = toDateObject(value);
  16|   if (!dateValue) {
  17|     return "Pick date";
  18|   }
  19|   return dateValue.toLocaleDateString();
  20| };
  21|
  22| export const DatePickerField = ({ id, label, value, onChange, required = false, testIdPrefix }) => {
  23|   return (
  24|     <div className="space-y-2" data-testid={`${testIdPrefix}-wrapper`}>
  25|       <label htmlFor={id} className="text-sm font-medium" data-testid={`${testIdPrefix}-label`}>
  26|         {label}
  27|       </label>
  28|       <Popover>
  29|         <PopoverTrigger asChild>
  30|           <Button
  31|             id={id}
  32|             type="button"
  33|             className="w-full justify-between rounded-none border-2 border-[#18181b] bg-white text-[#18181b]
hover:bg-[#18181b] hover:text-white"
  34|             data-testid={`${testIdPrefix}-button`}
  35|           >
  36|             <span data-testid={`${testIdPrefix}-value`}>{toDisplayValue(value)}</span>
  37|             <CalendarIcon size={16} />
  38|           </Button>
  39|         </PopoverTrigger>
  40|         <PopoverContent className="w-auto border-2 border-[#18181b] bg-white p-0" align="start"
data-testid={`${testIdPrefix}-popover`}>
  41|           <Calendar
  42|             mode="single"
  43|             selected={toDateObject(value)}
  44|             onSelect={(date) => {
  45|               if (!date) {
  46|                 if (!required) {
  47|                   onChange("");
  48|                 }
  49|                 return;
  50|               }
  51|               onChange(date.toISOString().slice(0, 10));
  52|             }}
  53|             initialFocus
  54|             data-testid={`${testIdPrefix}-calendar`}
  55|           />
  56|         </PopoverContent>
  57|       </Popover>
  58|     </div>
  59|   );
  60| };
```

FILE: /app/frontend/src/pages/DashboardPage.jsx

```
   1| import { Link } from "react-router-dom";
   2| import { motion } from "framer-motion";
   3| import { ArrowRight, CheckCircle2, Clock3, Sparkles } from "lucide-react";
   4| import { Button } from "@/components/ui/button";
   5| import { BrutalCard } from "@/components/BrutalCard";
   6|
   7| const DASHBOARD_MOTION_PROPS = {
   8|   initial: { opacity: 0, y: 20 },
   9|   animate: { opacity: 1, y: 0 },
```

```
  transition: { duration: 0.35 },
};

const toReadableDate = (value) => {
  if (!value) {
    return "No date";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return value;
  }
  return parsed.toLocaleDateString();
};

export default function DashboardPage({ tasks, deadlines, analytics, plans }) {
  const pendingTasks = tasks.filter((task) => !task.completed).slice(0, 4);
  const upcomingDeadlines = deadlines.filter((deadline) => !deadline.completed).slice(0, 5);
  const latestPlan = plans[0];

  return (
    <section className="space-y-8" data-testid="dashboard-page-root">
      <motion.div className="grid grid-cols-1 gap-8 lg:grid-cols-3" {...DASHBOARD_MOTION_PROPS}>
        <div className="brutal-card card-appear overflow-hidden border-2 border-[#18181b] bg-white lg:col-span-2"
data-testid="dashboard-hero-card">
          <div className="grid gap-0 md:grid-cols-2">
            <div className="space-y-5 p-7">
              <p className="text-sm font-medium uppercase tracking-wider text-zinc-600" data-testid="dashboard-hero-label">
                Your daily sanctuary
              </p>
              <h2 className="text-4xl sm:text-5xl lg:text-6xl" data-testid="dashboard-hero-title">
                Calm plan. <br /> Bold progress.
              </h2>
              <p className="text-sm md:text-base" data-testid="dashboard-hero-description">
                Build your AI timetable, execute focused blocks, and stay ahead of every deadline.
              </p>
              <Button
                asChild
                className="rounded-full border-2 border-[#18181b] bg-[#18181b] px-7 py-6 font-bold text-white
transition-transform hover:scale-105"
                data-testid="dashboard-open-planner-button"
              >
                <Link to="/planner">
                  Open AI Planner <ArrowRight className="ml-1" size={16} />
                </Link>
              </Button>
            </div>
            <div className="relative aspect-[4/3] md:aspect-auto" data-testid="dashboard-hero-image-wrap">
              <img
                src="https://images.unsplash.com/photo-1650661926447-9efb2610f64c?crop=entropy&cs=srgb&fm=jpg&ixid=M3w4NjA1N
zV8MHwxfHNlYXJjaHwxfHxhZXN0aGV0aWMlMjBzdHVkeSUyMGRlc2slMjBzZXR1cCUyMHN0dWRlbnR8ZW58MHx8fHwxNzczNjQyMDcwfDA&ixlib=rb-4.1.0&q=85"
                alt="Study desk with planner"
                className="h-full w-full object-cover object-center"
                data-testid="dashboard-hero-image"
              />
            </div>
          </div>
        </div>

        <BrutalCard
          className="bg-[#fecaca]"
          title="Today at a glance"
          subtitle="Core metrics"
          cardTestId="dashboard-summary-card"
        >
          <div className="space-y-4 text-sm md:text-base">
            <div className="flex items-center justify-between" data-testid="dashboard-metric-pending-tasks-row">
              <span>Pending tasks</span>
              <strong data-testid="dashboard-metric-pending-tasks-value">{pendingTasks.length}</strong>
            </div>
            <div className="flex items-center justify-between" data-testid="dashboard-metric-focus-minutes-row">
              <span>Focus minutes</span>
              <strong data-testid="dashboard-metric-focus-minutes-value">{analytics.focus_minutes_total}</strong>
            </div>
            <div className="flex items-center justify-between" data-testid="dashboard-metric-score-row">
              <span>Productivity score</span>
              <strong data-testid="dashboard-metric-score-value">{analytics.productivity_score}%</strong>
            </div>
          </div>
        </BrutalCard>
      </motion.div>

      <div className="grid grid-cols-1 gap-6 md:grid-cols-4" data-testid="dashboard-bento-grid">
        <BrutalCard className="bg-[#fecaca] md:col-span-1" title="Focus" subtitle="Pomodoro readiness"
cardTestId="dashboard-focus-card">
```

```
  90|           <div className="space-y-3">
  91|             <p className="text-sm" data-testid="dashboard-focus-text">Start a 25-minute sprint and log your session.</p>
  92|             <Button
  93|               asChild
  94|               className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b] hover:text-white"
  95|               data-testid="dashboard-go-focus-button"
  96|             >
  97|               <Link to="/focus">Go to Focus</Link>
  98|             </Button>
  99|           </div>
 100|         </BrutalCard>
 101|
 102|         <BrutalCard className="bg-white md:col-span-2" title="Today's Plan" subtitle="From your latest AI schedule"
cardTestId="dashboard-latest-plan-card">
 103|           <div className="space-y-3" data-testid="dashboard-latest-plan-content">
 104|             {latestPlan ? (
 105|               latestPlan.generated_schedule.slice(0, 2).map((day, index) => (
 106|                 <div className="border-b-2 border-[#18181b] pb-3" key={`${day.day}-${index}`}
data-testid={`dashboard-plan-day-${index}`}>
 107|                   <p className="text-sm font-semibold" data-testid={`dashboard-plan-day-${index}-name`}>{day.day}</p>
 108|                   <p className="text-sm" data-testid={`dashboard-plan-day-${index}-topics`}>{day.topics.join(", ")}</p>
 109|                 </div>
 110|               ))
 111|             ) : (
 112|               <p className="text-sm" data-testid="dashboard-empty-plan-message">No AI plan yet. Generate one in AI
Planner.</p>
 113|             )}
 114|           </div>
 115|         </BrutalCard>
 116|
 117|         <BrutalCard className="bg-[#bbf7d0] md:col-span-1" title="Progress" subtitle="Task completion"
cardTestId="dashboard-progress-card">
 118|           <div className="space-y-2">
 119|             <p className="text-sm" data-testid="dashboard-progress-count">
 120|               {analytics.tasks_completed} / {analytics.tasks_total} tasks done
 121|             </p>
 122|             <div className="h-3 w-full border-2 border-[#18181b] bg-white" data-testid="dashboard-progress-bar-track">
 123|               <div
 124|                 className="h-full bg-[#18181b]"
 125|                 style={{ width: `${analytics.tasks_total ? (analytics.tasks_completed / analytics.tasks_total) * 100 : 0}%`
}}
 126|                 data-testid="dashboard-progress-bar-fill"
 127|               />
 128|             </div>
 129|           </div>
 130|         </BrutalCard>
 131|       </div>
 132|
 133|       <BrutalCard
 134|         className="bg-[#bfdbfe]"
 135|         title="Upcoming Deadlines"
 136|         subtitle="Stay ahead of critical dates"
 137|         cardTestId="dashboard-deadline-card"
 138|       >
 139|         <div className="space-y-3" data-testid="dashboard-deadline-list">
 140|           {upcomingDeadlines.length === 0 ? (
 141|             <p className="text-sm" data-testid="dashboard-deadline-empty">No upcoming deadlines right now.</p>
 142|           ) : (
 143|             upcomingDeadlines.map((deadline, index) => (
 144|               <div
 145|                 className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
 146|                 key={deadline.id}
 147|                 data-testid={`dashboard-deadline-item-${index}`}
 148|               >
 149|                 <div>
 150|                   <p className="font-medium" data-testid={`dashboard-deadline-item-${index}-title`}>{deadline.title}</p>
 151|                   <p className="text-sm" data-testid={`dashboard-deadline-item-${index}-date`}>
 152|                     Due {toReadableDate(deadline.due_date)}
 153|                   </p>
 154|                 </div>
 155|                 <div className="inline-flex items-center gap-2 text-sm"
data-testid={`dashboard-deadline-item-${index}-status`}>
 156|                   {deadline.completed ? <CheckCircle2 size={16} /> : <Clock3 size={16} />}
 157|                   {deadline.completed ? "Done" : "Active"}
 158|                 </div>
 159|               </div>
 160|             ))
 161|           )}
 162|         </div>
 163|         <div className="mt-5 flex flex-wrap gap-3">
 164|           <Button
 165|             asChild
 166|             className="rounded-full border-2 border-[#18181b] bg-[#18181b] px-7 py-6 font-bold text-white
transition-transform hover:scale-105"
```

```
167|             data-testid="dashboard-open-deadlines-button"
168|           >
169|             <Link to="/deadlines">Manage Deadlines</Link>
170|           </Button>
171|           <Button
172|             asChild
173|             variant="outline"
174|             className="rounded-full border-2 border-[#18181b] bg-white px-7 py-6 font-bold text-[#18181b] hover:bg-[#18181b]
hover:text-white"
175|             data-testid="dashboard-open-analytics-button"
176|           >
177|             <Link to="/analytics">
178|               View Analytics <Sparkles className="ml-1" size={16} />
179|             </Link>
180|           </Button>
181|         </div>
182|       </BrutalCard>
183|     </section>
184|   );
185| }
```

FILE: /app/frontend/src/pages/PlannerPage.jsx

```
  1| import { useState } from "react";
  2| import { motion } from "framer-motion";
  3| import { WandSparkles } from "lucide-react";
  4| import { Button } from "@/components/ui/button";
  5| import { Input } from "@/components/ui/input";
  6| import { Textarea } from "@/components/ui/textarea";
  7| import { BrutalCard } from "@/components/BrutalCard";
  8|
  9| const DEFAULT_AVAILABLE_HOURS = 3;
 10| const DEFAULT_STUDY_DAYS = 7;
 11| const PAGE_MOTION_PROPS = {
 12|   initial: { opacity: 0, y: 20 },
 13|   animate: { opacity: 1, y: 0 },
 14|   transition: { duration: 0.35 },
 15| };
 16|
 17| export default function PlannerPage({ generatingPlan, latestPlan, onGeneratePlan }) {
 18|   const [goals, setGoals] = useState("");
 19|   const [priorities, setPriorities] = useState("");
 20|   const [availableHours, setAvailableHours] = useState(DEFAULT_AVAILABLE_HOURS);
 21|   const [studyDays, setStudyDays] = useState(DEFAULT_STUDY_DAYS);
 22|
 23|   const handleSubmit = async (event) => {
 24|     event.preventDefault();
 25|     await onGeneratePlan({
 26|       goals,
 27|       priorities,
 28|       available_hours_per_day: Number(availableHours),
 29|       study_days: Number(studyDays),
 30|     });
 31|   };
 32|
 33|   return (
 34|     <section className="space-y-8" data-testid="planner-page-root">
 35|       <motion.div {...PAGE_MOTION_PROPS} className="grid grid-cols-1 gap-6 xl:grid-cols-2">
 36|         <BrutalCard
 37|           className="bg-[#e9d5ff]"
 38|           title="AI Study Timetable Generator"
 39|           subtitle="Describe your goal and get a practical, day-by-day plan."
 40|           cardTestId="planner-form-card"
 41|         >
 42|           <form className="space-y-6" onSubmit={handleSubmit} data-testid="planner-form">
 43|             <div className="space-y-2">
 44|               <label htmlFor="planner-goals" className="text-sm font-medium" data-testid="planner-goals-label">
 45|                 Study Goals
 46|               </label>
 47|               <Textarea
 48|                 id="planner-goals"
 49|                 value={goals}
 50|                 onChange={(event) => setGoals(event.target.value)}
 51|                 placeholder="Example: Math finals, organic chemistry revision, and essay writing practice"
 52|                 className="min-h-24 border-2 border-[#18181b] rounded-none"
 53|                 required
 54|                 data-testid="planner-goals-input"
 55|               />
 56|             </div>
 57|
 58|             <div className="space-y-2">
 59|               <label htmlFor="planner-priorities" className="text-sm font-medium" data-testid="planner-priorities-label">
 60|                 Priorities
 61|               </label>
 62|               <Input
```

```
  63|                id="planner-priorities"
  64|                value={priorities}
  65|                onChange={(event) => setPriorities(event.target.value)}
  66|                placeholder="Weak topics first, daily revision, practice questions"
  67|                className="border-2 border-[#18181b] rounded-none"
  68|                data-testid="planner-priorities-input"
  69|              />
  70|            </div>
  71|
  72|            <div className="grid grid-cols-1 gap-5 md:grid-cols-2">
  73|              <div className="space-y-2">
  74|                <label htmlFor="planner-hours" className="text-sm font-medium" data-testid="planner-hours-label">
  75|                  Hours available per day
  76|                </label>
  77|                <Input
  78|                  id="planner-hours"
  79|                  type="number"
  80|                  min="1"
  81|                  max="12"
  82|                  value={availableHours}
  83|                  onChange={(event) => setAvailableHours(event.target.value)}
  84|                  className="border-2 border-[#18181b] rounded-none"
  85|                  data-testid="planner-hours-input"
  86|                />
  87|              </div>
  88|              <div className="space-y-2">
  89|                <label htmlFor="planner-days" className="text-sm font-medium" data-testid="planner-days-label">
  90|                  Study days
  91|                </label>
  92|                <Input
  93|                  id="planner-days"
  94|                  type="number"
  95|                  min="3"
  96|                  max="30"
  97|                  value={studyDays}
  98|                  onChange={(event) => setStudyDays(event.target.value)}
  99|                  className="border-2 border-[#18181b] rounded-none"
 100|                  data-testid="planner-days-input"
 101|                />
 102|              </div>
 103|            </div>
 104|
 105|            <Button
 106|              type="submit"
 107|              className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
 108|              disabled={generatingPlan || !goals.trim()}
 109|              data-testid="planner-generate-button"
 110|            >
 111|              <WandSparkles className="mr-2" size={16} />
 112|              {generatingPlan ? "Generating..." : "Generate AI Study Plan"}
 113|            </Button>
 114|          </form>
 115|        </BrutalCard>
 116|
 117|        <BrutalCard
 118|          className="bg-white"
 119|          title="Latest Generated Plan"
 120|          subtitle="Use this as your daily execution roadmap."
 121|          cardTestId="planner-result-card"
 122|        >
 123|          {latestPlan ? (
 124|            <div className="space-y-4" data-testid="planner-generated-plan-wrap">
 125|              <p className="text-sm md:text-base" data-testid="planner-generated-plan-summary">{latestPlan.summary}</p>
 126|              {latestPlan.generated_schedule.map((day, index) => (
 127|                <div className="border-b-2 border-[#18181b] pb-3" key={`${day.day}-${index}`}
data-testid={`planner-day-item-${index}`}>
 128|                  <p className="font-medium" data-testid={`planner-day-item-${index}-name`}>{day.day}</p>
 129|                  <p className="text-sm" data-testid={`planner-day-item-${index}-topics`}>
 130|                    Topics: {day.topics.join(", ")}
 131|                  </p>
 132|                  <p className="text-sm" data-testid={`planner-day-item-${index}-minutes`}>
 133|                    Focus: {day.focus_minutes} min
 134|                  </p>
 135|                  <p className="text-sm" data-testid={`planner-day-item-${index}-notes`}>{day.notes}</p>
 136|                </div>
 137|              ))}
 138|            </div>
 139|          ) : (
 140|            <p className="text-sm" data-testid="planner-empty-message">
 141|              No plan yet. Fill the form and generate your first AI timetable.
 142|            </p>
 143|          )}
 144|        </BrutalCard>
```

```
      </motion.div>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/TasksPage.jsx

```
import { useState } from "react";
import { motion } from "framer-motion";
import { Check, Pencil, Plus, Save, Trash2, X } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { BrutalCard } from "@/components/BrutalCard";
import { DatePickerField } from "@/components/DatePickerField";

const DEFAULT_ESTIMATED_MINUTES = 45;
const ACTION_ICON_SIZE = 15;
const TASK_STAGGER_DELAY = 0.05;
const TASK_ITEM_VARIANTS = {
  hidden: { opacity: 0, y: 8 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * TASK_STAGGER_DELAY },
  }),
};

const toDisplayDate = (value) => {
  if (!value) {
    return "No due date";
  }
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleDateString();
};

const toDateInput = (value) => {
  if (!value) {
    return "";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return "";
  }
  return parsed.toISOString().slice(0, 10);
};

export default function TasksPage({
  submittingTask,
  tasks,
  onAddTask,
  onToggleTask,
  onDeleteTask,
  onUpdateTask,
}) {
  const [title, setTitle] = useState("");
  const [subject, setSubject] = useState("General");
  const [priority, setPriority] = useState("medium");
  const [dueDate, setDueDate] = useState("");
  const [estimatedMinutes, setEstimatedMinutes] = useState(DEFAULT_ESTIMATED_MINUTES);
  const [editingTaskId, setEditingTaskId] = useState("");
  const [taskDraft, setTaskDraft] = useState({
    title: "",
    subject: "General",
    priority: "medium",
    dueDate: "",
    estimatedMinutes: DEFAULT_ESTIMATED_MINUTES,
  });

  const submitTask = async (event) => {
    event.preventDefault();
    await onAddTask({
      title,
      subject,
      priority,
      due_date: dueDate ? new Date(dueDate).toISOString() : null,
      estimated_minutes: Number(estimatedMinutes),
      completed: false,
    });
    setTitle("");
    setSubject("General");
    setPriority("medium");
    setDueDate("");
    setEstimatedMinutes(DEFAULT_ESTIMATED_MINUTES);
  };

```

```
  79|   const startTaskEdit = (task) => {
  80|     setEditingTaskId(task.id);
  81|     setTaskDraft({
  82|       title: task.title,
  83|       subject: task.subject,
  84|       priority: task.priority,
  85|       dueDate: toDateInput(task.due_date),
  86|       estimatedMinutes: task.estimated_minutes,
  87|     });
  88|   };
  89|
  90|   const cancelTaskEdit = () => {
  91|     setEditingTaskId("");
  92|   };
  93|
  94|   const saveTaskEdit = async () => {
  95|     if (!editingTaskId || !taskDraft.title.trim()) {
  96|       return;
  97|     }
  98|     await onUpdateTask(editingTaskId, {
  99|       title: taskDraft.title,
 100|       subject: taskDraft.subject,
 101|       priority: taskDraft.priority,
 102|       due_date: taskDraft.dueDate ? new Date(taskDraft.dueDate).toISOString() : null,
 103|       estimated_minutes: Number(taskDraft.estimatedMinutes),
 104|     });
 105|     setEditingTaskId("");
 106|   };
 107|
 108|   return (
 109|     <section className="space-y-8" data-testid="tasks-page-root">
 110|       <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
 111|         <BrutalCard
 112|           className="bg-[#bfdbfe]"
 113|           title="Create Daily Task"
 114|           subtitle="Quickly add what needs to get done."
 115|           cardTestId="task-create-card"
 116|         >
 117|           <form className="space-y-5" onSubmit={submitTask} data-testid="task-create-form">
 118|             <div className="space-y-2">
 119|               <label htmlFor="task-title" className="text-sm font-medium" data-testid="task-title-label">Task title</label>
 120|               <Input
 121|                 id="task-title"
 122|                 value={title}
 123|                 onChange={(event) => setTitle(event.target.value)}
 124|                 required
 125|                 placeholder="Solve chapter 5 practice questions"
 126|                 className="border-2 border-[#18181b] rounded-none"
 127|                 data-testid="task-title-input"
 128|               />
 129|             </div>
 130|
 131|             <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
 132|               <div className="space-y-2">
 133|                 <label htmlFor="task-subject" className="text-sm font-medium"
data-testid="task-subject-label">Subject</label>
 134|                 <Input
 135|                   id="task-subject"
 136|                   value={subject}
 137|                   onChange={(event) => setSubject(event.target.value)}
 138|                   className="border-2 border-[#18181b] rounded-none"
 139|                   data-testid="task-subject-input"
 140|                 />
 141|               </div>
 142|               <div className="space-y-2">
 143|                 <label htmlFor="task-priority" className="text-sm font-medium"
data-testid="task-priority-label">Priority</label>
 144|                 <select
 145|                   id="task-priority"
 146|                   value={priority}
 147|                   onChange={(event) => setPriority(event.target.value)}
 148|                   className="h-10 w-full border-2 border-[#18181b] bg-white px-3 text-sm"
 149|                   data-testid="task-priority-select"
 150|                 >
 151|                   <option value="low">Low</option>
 152|                   <option value="medium">Medium</option>
 153|                   <option value="high">High</option>
 154|                 </select>
 155|               </div>
 156|             </div>
 157|
 158|             <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
 159|               <DatePickerField
 160|                 id="task-due-date"
```

```
161|                label="Due date"
162|                value={dueDate}
163|                onChange={setDueDate}
164|                testIdPrefix="task-due-date"
165|              />
166|              <div className="space-y-2">
167|                <label htmlFor="task-estimate" className="text-sm font-medium" data-testid="task-estimate-label">
168|                  Estimated minutes
169|                </label>
170|                <Input
171|                  id="task-estimate"
172|                  type="number"
173|                  min="5"
174|                  max="240"
175|                  value={estimatedMinutes}
176|                  onChange={(event) => setEstimatedMinutes(event.target.value)}
177|                  className="border-2 border-[#18181b] rounded-none"
178|                  data-testid="task-estimate-input"
179|                />
180|              </div>
181|            </div>
182|
183|            <Button
184|              type="submit"
185|              disabled={submittingTask || !title.trim()}
186|              className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
187|              data-testid="task-submit-button"
188|            >
189|              <Plus className="mr-2" size={16} />
190|              {submittingTask ? "Adding..." : "Add Task"}
191|            </Button>
192|          </form>
193|        </BrutalCard>
194|
195|        <BrutalCard
196|          className="bg-white"
197|          title="Task Queue"
198|          subtitle="Check off tasks as you complete them."
199|          cardTestId="task-list-card"
200|        >
201|          <div className="space-y-3" data-testid="task-list-wrap">
202|            {tasks.length === 0 ? (
203|              <p className="text-sm" data-testid="task-empty-message">No tasks yet. Add your first task.</p>
204|            ) : (
205|              tasks.map((task, index) => (
206|                <motion.div
207|                  key={task.id}
208|                  custom={index}
209|                  variants={TASK_ITEM_VARIANTS}
210|                  initial="hidden"
211|                  animate="visible"
212|                  className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
213|                  data-testid={`task-item-${index}`}
214|                >
215|                  {editingTaskId === task.id ? (
216|                    <div className="w-full space-y-4" data-testid={`task-item-${index}-edit-wrap`}>
217|                      <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
218|                        <div className="space-y-2">
219|                          <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-title-label`}>
220|                            Title
221|                          </label>
222|                          <Input
223|                            value={taskDraft.title}
224|                            onChange={(event) => setTaskDraft((prev) => ({ ...prev, title: event.target.value }))}
225|                            className="border-2 border-[#18181b] rounded-none"
226|                            data-testid={`task-item-${index}-edit-title-input`}
227|                          />
228|                        </div>
229|                        <div className="space-y-2">
230|                          <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-subject-label`}>
231|                            Subject
232|                          </label>
233|                          <Input
234|                            value={taskDraft.subject}
235|                            onChange={(event) => setTaskDraft((prev) => ({ ...prev, subject: event.target.value }))}
236|                            className="border-2 border-[#18181b] rounded-none"
237|                            data-testid={`task-item-${index}-edit-subject-input`}
238|                          />
239|                        </div>
240|                      </div>
241|                      <div className="grid grid-cols-1 gap-3 md:grid-cols-3">
242|                        <div className="space-y-2">
243|                          <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-priority-label`}>
```

```
244|                           Priority
245|                         </label>
246|                         <select
247|                           value={taskDraft.priority}
248|                           onChange={(event) => setTaskDraft((prev) => ({ ...prev, priority: event.target.value }))}
249|                           className="h-10 w-full border-2 border-[#18181b] bg-white px-3 text-sm"
250|                           data-testid={`task-item-${index}-edit-priority-select`}
251|                         >
252|                           <option value="low">Low</option>
253|                           <option value="medium">Medium</option>
254|                           <option value="high">High</option>
255|                         </select>
256|                       </div>
257|                       <DatePickerField
258|                         id={`task-edit-due-date-${index}`}
259|                         label="Due date"
260|                         value={taskDraft.dueDate}
261|                         onChange={(value) => setTaskDraft((prev) => ({ ...prev, dueDate: value }))}
262|                         testIdPrefix={`task-item-${index}-edit-due-date`}
263|                       />
264|                       <div className="space-y-2">
265|                         <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-estimate-label`}>
266|                           Estimated minutes
267|                         </label>
268|                         <Input
269|                           type="number"
270|                           min="5"
271|                           max="240"
272|                           value={taskDraft.estimatedMinutes}
273|                           onChange={(event) =>
274|                             setTaskDraft((prev) => ({ ...prev, estimatedMinutes: event.target.value }))
275|                           }
276|                           className="border-2 border-[#18181b] rounded-none"
277|                           data-testid={`task-item-${index}-edit-estimate-input`}
278|                         />
279|                       </div>
280|                     </div>
281|                     <div className="flex flex-wrap gap-2">
282|                       <Button
283|                         type="button"
284|                         onClick={saveTaskEdit}
285|                         className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
286|                         data-testid={`task-item-${index}-edit-save-button`}
287|                       >
288|                         <Save size={ACTION_ICON_SIZE} className="mr-1" />
289|                         Save
290|                       </Button>
291|                       <Button
292|                         type="button"
293|                         onClick={cancelTaskEdit}
294|                         className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
295|                         data-testid={`task-item-${index}-edit-cancel-button`}
296|                       >
297|                         <X size={ACTION_ICON_SIZE} className="mr-1" />
298|                         Cancel
299|                       </Button>
300|                     </div>
301|                   </div>
302|                 ) : (
303|                   <>
304|                     <div className="space-y-1">
305|                       <p
306|                         className={`font-medium ${task.completed ? "line-through text-zinc-500" : ""}`}
307|                         data-testid={`task-item-${index}-title`}
308|                       >
309|                         {task.title}
310|                       </p>
311|                       <p className="text-sm" data-testid={`task-item-${index}-meta`}>
312|                         {task.subject} • {task.priority} • {toDisplayDate(task.due_date)} • {task.estimated_minutes} min
313|                       </p>
314|                     </div>
315|                     <div className="flex gap-2">
316|                       <Button
317|                         type="button"
318|                         onClick={() => startTaskEdit(task)}
319|                         className="rounded-full border-2 border-[#18181b] bg-[#e9d5ff] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
320|                         data-testid={`task-item-${index}-edit-button`}
321|                       >
322|                         <Pencil size={ACTION_ICON_SIZE} />
323|                       </Button>
324|                       <Button
325|                         type="button"
```

```
 326|                          onClick={() => onToggleTask(task)}
 327|                          className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
 328|                          data-testid={`task-item-${index}-toggle-button`}
 329|                        >
 330|                          <Check size={ACTION_ICON_SIZE} />
 331|                        </Button>
 332|                        <Button
 333|                          type="button"
 334|                          onClick={() => onDeleteTask(task.id)}
 335|                          className="rounded-full border-2 border-[#18181b] bg-[#fecaca] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
 336|                          data-testid={`task-item-${index}-delete-button`}
 337|                        >
 338|                          <Trash2 size={ACTION_ICON_SIZE} />
 339|                        </Button>
 340|                      </div>
 341|                    </>
 342|                  )}
 343|                </motion.div>
 344|              ))
 345|            )}
 346|          </div>
 347|        </BrutalCard>
 348|      </div>
 349|    </section>
 350|  );
 351| }
```

FILE: /app/frontend/src/pages/FocusPage.jsx

```
   1| import { useEffect, useMemo, useState } from "react";
   2| import { motion } from "framer-motion";
   3| import { Pause, Play, RotateCcw, Save } from "lucide-react";
   4| import { Button } from "@/components/ui/button";
   5| import { BrutalCard } from "@/components/BrutalCard";
   6| import { toast } from "@/components/ui/sonner";
   7|
   8| const formatSeconds = (value) => {
   9|   const minutes = Math.floor(value / 60);
  10|   const seconds = value % 60;
  11|   return `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(2, "0")}`;
  12| };
  13|
  14| const formatSessionDate = (value) => {
  15|   const parsed = new Date(value);
  16|   return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleString();
  17| };
  18|
  19| const DEFAULT_FOCUS_MINUTES = 25;
  20| const FOCUS_MODE_OPTIONS = [25, 50];
  21| const CONTROL_ICON_SIZE = 16;
  22| const STAGGER_DELAY = 0.05;
  23| const HISTORY_ITEM_VARIANTS = {
  24|   hidden: { opacity: 0, y: 8 },
  25|   visible: (index) => ({
  26|     opacity: 1,
  27|     y: 0,
  28|     transition: { delay: index * STAGGER_DELAY },
  29|   }),
  30| };
  31|
  32| export default function FocusPage({ focusSessions, onLogSession }) {
  33|   const [modeMinutes, setModeMinutes] = useState(DEFAULT_FOCUS_MINUTES);
  34|   const [secondsLeft, setSecondsLeft] = useState(DEFAULT_FOCUS_MINUTES * 60);
  35|   const [isRunning, setIsRunning] = useState(false);
  36|
  37|   const finishedMinutes = useMemo(() => Math.round((modeMinutes * 60 - secondsLeft) / 60), [modeMinutes, secondsLeft]);
  38|
  39|   useEffect(() => {
  40|     if (!isRunning) {
  41|       return undefined;
  42|     }
  43|
  44|     const interval = setInterval(() => {
  45|       setSecondsLeft((previous) => {
  46|         if (previous <= 1) {
  47|           setIsRunning(false);
  48|           toast.success("Pomodoro complete! Session logged.");
  49|           onLogSession(modeMinutes);
  50|           return 0;
  51|         }
  52|         return previous - 1;
  53|       });
  54|     }, 1000);
```

```
  55|
  56|     return () => clearInterval(interval);
  57|   }, [isRunning, modeMinutes, onLogSession]);
  58|
  59|   const switchMode = (minutes) => {
  60|     setModeMinutes(minutes);
  61|     setSecondsLeft(minutes * 60);
  62|     setIsRunning(false);
  63|   };
  64|
  65|   const resetTimer = () => {
  66|     setSecondsLeft(modeMinutes * 60);
  67|     setIsRunning(false);
  68|   };
  69|
  70|   const saveCurrentProgress = async () => {
  71|     if (!finishedMinutes) {
  72|       toast.info("Start the timer to log focus minutes.");
  73|       return;
  74|     }
  75|     await onLogSession(finishedMinutes);
  76|     resetTimer();
  77|   };
  78|
  79|   return (
  80|     <section className="space-y-8" data-testid="focus-page-root">
  81|       <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
  82|         <BrutalCard
  83|           className="bg-[#fecaca]"
  84|           title="Pomodoro Focus Timer"
  85|           subtitle="Stay intense, then recover quickly."
  86|           cardTestId="focus-timer-card"
  87|         >
  88|           <div className="space-y-6" data-testid="focus-timer-wrap">
  89|             <div className="flex flex-wrap gap-3" data-testid="focus-mode-buttons-wrap">
  90|               {FOCUS_MODE_OPTIONS.map((minutes) => (
  91|                 <Button
  92|                   key={minutes}
  93|                   type="button"
  94|                   onClick={() => switchMode(minutes)}
  95|                   className={`rounded-full border-2 border-[#18181b] ${
  96|                     modeMinutes === minutes ? "bg-[#18181b] text-white" : "bg-white text-[#18181b]"
  97|                   } hover:bg-[#18181b] hover:text-white`}
  98|                   data-testid={`focus-mode-${minutes}-button`}
  99|                 >
 100|                   {minutes} min mode
 101|                 </Button>
 102|               ))}
 103|             </div>
 104|
 105|             <p className="timer-font text-6xl font-semibold" data-testid="focus-timer-display">
 106|               {formatSeconds(secondsLeft)}
 107|             </p>
 108|
 109|             <div className="flex flex-wrap gap-3" data-testid="focus-controls-wrap">
 110|               <Button
 111|                 type="button"
 112|                 onClick={() => setIsRunning((previous) => !previous)}
 113|                 className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
 114|                 data-testid="focus-toggle-button"
 115|               >
 116|                 {isRunning ? <Pause size={CONTROL_ICON_SIZE} className="mr-2" /> : <Play size={CONTROL_ICON_SIZE}
className="mr-2" />}
 117|                 {isRunning ? "Pause" : "Start"}
 118|               </Button>
 119|               <Button
 120|                 type="button"
 121|                 onClick={resetTimer}
 122|                 className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
 123|                 data-testid="focus-reset-button"
 124|               >
 125|                 <RotateCcw size={CONTROL_ICON_SIZE} className="mr-2" />
 126|                 Reset
 127|               </Button>
 128|               <Button
 129|                 type="button"
 130|                 onClick={saveCurrentProgress}
 131|                 className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
 132|                 data-testid="focus-save-progress-button"
 133|               >
 134|                 <Save size={CONTROL_ICON_SIZE} className="mr-2" />
 135|                 Log {finishedMinutes || 0} min
```

```
              </Button>
            </div>
          </div>
        </BrutalCard>

        <BrutalCard className="bg-white" title="Recent Focus Sessions" subtitle="Your concentration history"
cardTestId="focus-history-card">
          <div className="space-y-3" data-testid="focus-history-list">
            {focusSessions.length === 0 ? (
              <p className="text-sm" data-testid="focus-history-empty">No sessions logged yet.</p>
            ) : (
              focusSessions.slice(0, 8).map((session, index) => (
                <motion.div
                  key={session.id}
                  custom={index}
                  variants={HISTORY_ITEM_VARIANTS}
                  initial="hidden"
                  animate="visible"
                  className="flex items-center justify-between border-b-2 border-[#18181b] pb-3"
                  data-testid={`focus-history-item-${index}`}
                >
                  <p className="text-sm" data-testid={`focus-history-item-${index}-date`}>
                    {formatSessionDate(session.started_at)}
                  </p>
                  <p className="text-sm font-medium" data-testid={`focus-history-item-${index}-minutes`}>
                    {session.duration_minutes} min
                  </p>
                </motion.div>
              ))
            )}
          </div>
        </BrutalCard>
      </div>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/DeadlinesPage.jsx

```
import { useState } from "react";
import { motion } from "framer-motion";
import { Bell, Check, Pencil, Save, Trash2, X } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { BrutalCard } from "@/components/BrutalCard";
import { DatePickerField } from "@/components/DatePickerField";

const formatDate = (value) => {
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleDateString();
};

const urgencyColor = (daysLeft) => {
  if (daysLeft < 0) {
    return "bg-[#fecaca]";
  }
  if (daysLeft <= 2) {
    return "bg-[#fed7aa]";
  }
  return "bg-[#bbf7d0]";
};

const ACTION_ICON_SIZE = 15;
const DEADLINE_STAGGER_DELAY = 0.05;
const DEADLINE_ITEM_VARIANTS = {
  hidden: { opacity: 0, y: 8 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * DEADLINE_STAGGER_DELAY },
  }),
};

const toDateInput = (value) => {
  if (!value) {
    return "";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return "";
  }
  return parsed.toISOString().slice(0, 10);
};

```

```
 47| export default function DeadlinesPage({
 48|   submittingDeadline,
 49|   deadlines,
 50|   reminders,
 51|   onAddDeadline,
 52|   onToggleDeadline,
 53|   onDeleteDeadline,
 54|   onUpdateDeadline,
 55| }) {
 56|   const [title, setTitle] = useState("");
 57|   const [subject, setSubject] = useState("General");
 58|   const [dueDate, setDueDate] = useState("");
 59|   const [reminderDate, setReminderDate] = useState("");
 60|   const [notes, setNotes] = useState("");
 61|   const [editingDeadlineId, setEditingDeadlineId] = useState("");
 62|   const [deadlineDraft, setDeadlineDraft] = useState({
 63|     title: "",
 64|     subject: "General",
 65|     dueDate: "",
 66|     reminderDate: "",
 67|     notes: "",
 68|   });
 69|
 70|   const submitDeadline = async (event) => {
 71|     event.preventDefault();
 72|     await onAddDeadline({
 73|       title,
 74|       subject,
 75|       due_date: new Date(dueDate).toISOString(),
 76|       reminder_date: reminderDate ? new Date(reminderDate).toISOString() : null,
 77|       notes,
 78|       completed: false,
 79|     });
 80|     setTitle("");
 81|     setSubject("General");
 82|     setDueDate("");
 83|     setReminderDate("");
 84|     setNotes("");
 85|   };
 86|
 87|   const startDeadlineEdit = (deadline) => {
 88|     setEditingDeadlineId(deadline.id);
 89|     setDeadlineDraft({
 90|       title: deadline.title,
 91|       subject: deadline.subject,
 92|       dueDate: toDateInput(deadline.due_date),
 93|       reminderDate: toDateInput(deadline.reminder_date),
 94|       notes: deadline.notes || "",
 95|     });
 96|   };
 97|
 98|   const cancelDeadlineEdit = () => {
 99|     setEditingDeadlineId("");
100|   };
101|
102|   const saveDeadlineEdit = async () => {
103|     if (!editingDeadlineId || !deadlineDraft.title.trim() || !deadlineDraft.dueDate) {
104|       return;
105|     }
106|
107|     await onUpdateDeadline(editingDeadlineId, {
108|       title: deadlineDraft.title,
109|       subject: deadlineDraft.subject,
110|       due_date: new Date(deadlineDraft.dueDate).toISOString(),
111|       reminder_date: deadlineDraft.reminderDate ? new Date(deadlineDraft.reminderDate).toISOString() : null,
112|       notes: deadlineDraft.notes,
113|     });
114|     setEditingDeadlineId("");
115|   };
116|
117|   return (
118|     <section className="space-y-8" data-testid="deadlines-page-root">
119|       <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
120|         <BrutalCard
121|           className="bg-[#fed7aa]"
122|           title="Add Deadline"
123|           subtitle="Track exams, submissions, and milestones."
124|           cardTestId="deadline-form-card"
125|         >
126|           <form className="space-y-5" onSubmit={submitDeadline} data-testid="deadline-form">
127|             <div className="space-y-2">
128|               <label htmlFor="deadline-title" className="text-sm font-medium"
data-testid="deadline-title-label">Title</label>
129|               <Input
```

```
 130|                 id="deadline-title"
 131|                 value={title}
 132|                 onChange={(event) => setTitle(event.target.value)}
 133|                 required
 134|                 placeholder="Math final exam"
 135|                 className="border-2 border-[#18181b] rounded-none"
 136|                 data-testid="deadline-title-input"
 137|               />
 138|             </div>
 139|
 140|             <div className="space-y-2">
 141|               <label htmlFor="deadline-subject" className="text-sm font-medium"
data-testid="deadline-subject-label">Subject</label>
 142|               <Input
 143|                 id="deadline-subject"
 144|                 value={subject}
 145|                 onChange={(event) => setSubject(event.target.value)}
 146|                 className="border-2 border-[#18181b] rounded-none"
 147|                 data-testid="deadline-subject-input"
 148|               />
 149|             </div>
 150|
 151|             <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
 152|               <DatePickerField
 153|                 id="deadline-due-date"
 154|                 label="Due date"
 155|                 value={dueDate}
 156|                 onChange={setDueDate}
 157|                 required
 158|                 testIdPrefix="deadline-due-date"
 159|               />
 160|               <DatePickerField
 161|                 id="deadline-reminder-date"
 162|                 label="Reminder date"
 163|                 value={reminderDate}
 164|                 onChange={setReminderDate}
 165|                 testIdPrefix="deadline-reminder-date"
 166|               />
 167|             </div>
 168|
 169|             <div className="space-y-2">
 170|               <label htmlFor="deadline-notes" className="text-sm font-medium"
data-testid="deadline-notes-label">Notes</label>
 171|               <Textarea
 172|                 id="deadline-notes"
 173|                 value={notes}
 174|                 onChange={(event) => setNotes(event.target.value)}
 175|                 className="min-h-20 border-2 border-[#18181b] rounded-none"
 176|                 data-testid="deadline-notes-input"
 177|               />
 178|             </div>
 179|
 180|             <Button
 181|               type="submit"
 182|               disabled={submittingDeadline || !title.trim() || !dueDate}
 183|               className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
 184|               data-testid="deadline-submit-button"
 185|             >
 186|               {submittingDeadline ? "Adding..." : "Save Deadline"}
 187|             </Button>
 188|           </form>
 189|         </BrutalCard>
 190|
 191|         <BrutalCard className="bg-white" title="Deadline Alerts" subtitle="Critical reminders from your timeline"
cardTestId="deadline-reminder-card">
 192|           <div className="space-y-3" data-testid="deadline-reminder-list">
 193|             {reminders.length === 0 ? (
 194|               <p className="text-sm" data-testid="deadline-reminder-empty">No urgent reminders now.</p>
 195|             ) : (
 196|               reminders.map((item, index) => (
 197|                 <div
 198|                   className={`flex items-center justify-between border-2 border-[#18181b] p-3
${urgencyColor(item.days_left)}`}
 199|                   key={item.id}
 200|                   data-testid={`deadline-reminder-item-${index}`}
 201|                 >
 202|                   <div>
 203|                     <p className="font-medium" data-testid={`deadline-reminder-item-${index}-title`}>{item.title}</p>
 204|                     <p className="text-sm" data-testid={`deadline-reminder-item-${index}-days`}>
 205|                       {item.days_left < 0 ? `${Math.abs(item.days_left)} days overdue` : `${item.days_left} days left`}
 206|                     </p>
 207|                   </div>
 208|                   <Bell size={18} />
```

```
209|                </div>
210|              ))
211|            )}
212|          </div>
213|        </BrutalCard>
214|      </div>
215|
216|      <BrutalCard className="bg-[#bfdbfe]" title="All Deadlines" subtitle="Mark complete or remove as needed"
cardTestId="deadline-list-card">
217|        <div className="space-y-3" data-testid="deadline-list-wrap">
218|          {deadlines.length === 0 ? (
219|            <p className="text-sm" data-testid="deadline-empty-message">No deadlines yet.</p>
220|          ) : (
221|            deadlines.map((deadline, index) => (
222|              <motion.div
223|                key={deadline.id}
224|                custom={index}
225|                variants={DEADLINE_ITEM_VARIANTS}
226|                initial="hidden"
227|                animate="visible"
228|                className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
229|                data-testid={`deadline-item-${index}`}
230|              >
231|                {editingDeadlineId === deadline.id ? (
232|                  <div className="w-full space-y-4" data-testid={`deadline-item-${index}-edit-wrap`}>
233|                    <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
234|                      <div className="space-y-2">
235|                        <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-title-label`}>
236|                          Title
237|                        </label>
238|                        <Input
239|                          value={deadlineDraft.title}
240|                          onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, title: event.target.value }))}
241|                          className="border-2 border-[#18181b] rounded-none"
242|                          data-testid={`deadline-item-${index}-edit-title-input`}
243|                        />
244|                      </div>
245|                      <div className="space-y-2">
246|                        <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-subject-label`}>
247|                          Subject
248|                        </label>
249|                        <Input
250|                          value={deadlineDraft.subject}
251|                          onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, subject: event.target.value }))}
252|                          className="border-2 border-[#18181b] rounded-none"
253|                          data-testid={`deadline-item-${index}-edit-subject-input`}
254|                        />
255|                      </div>
256|                    </div>
257|                    <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
258|                      <DatePickerField
259|                        id={`deadline-edit-due-date-${index}`}
260|                        label="Due date"
261|                        value={deadlineDraft.dueDate}
262|                        onChange={(value) => setDeadlineDraft((prev) => ({ ...prev, dueDate: value }))}
263|                        required
264|                        testIdPrefix={`deadline-item-${index}-edit-due-date`}
265|                      />
266|                      <DatePickerField
267|                        id={`deadline-edit-reminder-date-${index}`}
268|                        label="Reminder date"
269|                        value={deadlineDraft.reminderDate}
270|                        onChange={(value) => setDeadlineDraft((prev) => ({ ...prev, reminderDate: value }))}
271|                        testIdPrefix={`deadline-item-${index}-edit-reminder-date`}
272|                      />
273|                    </div>
274|                    <div className="space-y-2">
275|                      <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-notes-label`}>
276|                        Notes
277|                      </label>
278|                      <Textarea
279|                        value={deadlineDraft.notes}
280|                        onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, notes: event.target.value }))}
281|                        className="min-h-20 border-2 border-[#18181b] rounded-none"
282|                        data-testid={`deadline-item-${index}-edit-notes-input`}
283|                      />
284|                    </div>
285|                    <div className="flex flex-wrap gap-2">
286|                      <Button
287|                        type="button"
288|                        onClick={saveDeadlineEdit}
289|                        className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
290|                        data-testid={`deadline-item-${index}-edit-save-button`}
291|                      >
```

```
                        <Save size={ACTION_ICON_SIZE} className="mr-1" />
                        Save
                      </Button>
                      <Button
                        type="button"
                        onClick={cancelDeadlineEdit}
                        className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-edit-cancel-button`}
                      >
                        <X size={ACTION_ICON_SIZE} className="mr-1" />
                        Cancel
                      </Button>
                    </div>
                  </div>
                ) : (
                  <>
                    <div>
                      <p className="font-medium" data-testid={`deadline-item-${index}-title`}>{deadline.title}</p>
                      <p className="text-sm" data-testid={`deadline-item-${index}-meta`}>
                        {deadline.subject} • Due {formatDate(deadline.due_date)}
                      </p>
                    </div>
                    <div className="flex gap-2">
                      <Button
                        type="button"
                        onClick={() => startDeadlineEdit(deadline)}
                        className="rounded-full border-2 border-[#18181b] bg-[#e9d5ff] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-edit-button`}
                      >
                        <Pencil size={ACTION_ICON_SIZE} />
                      </Button>
                      <Button
                        type="button"
                        onClick={() => onToggleDeadline(deadline)}
                        className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-toggle-button`}
                      >
                        <Check size={ACTION_ICON_SIZE} />
                      </Button>
                      <Button
                        type="button"
                        onClick={() => onDeleteDeadline(deadline.id)}
                        className="rounded-full border-2 border-[#18181b] bg-[#fecaca] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-delete-button`}
                      >
                        <Trash2 size={ACTION_ICON_SIZE} />
                      </Button>
                    </div>
                  </>
                )}
              </motion.div>
            ))
          )}
        </div>
      </BrutalCard>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/AnalyticsPage.jsx

```
import { motion } from "framer-motion";
import { BrutalCard } from "@/components/BrutalCard";

const metricCards = [
  { key: "tasks_total", label: "Total Tasks", bg: "bg-[#bfdbfe]" },
  { key: "tasks_completed", label: "Completed Tasks", bg: "bg-[#bbf7d0]" },
  { key: "focus_minutes_total", label: "Focus Minutes", bg: "bg-[#fecaca]" },
  { key: "productivity_score", label: "Productivity Score", bg: "bg-[#e9d5ff]", suffix: "%" },
];

const ANALYTICS_CARD_STAGGER = 0.05;
const ANALYTICS_CARD_VARIANTS = {
  hidden: { opacity: 0, y: 12 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * ANALYTICS_CARD_STAGGER },
  }),
};

```

```
export default function AnalyticsPage({ analytics }) {
  const completionRate = analytics.tasks_total
    ? Math.round((analytics.tasks_completed / analytics.tasks_total) * 100)
    : 0;

  return (
    <section className="space-y-8" data-testid="analytics-page-root">
      <div className="grid grid-cols-1 gap-6 md:grid-cols-2 xl:grid-cols-4" data-testid="analytics-metrics-grid">
        {metricCards.map((metric, index) => (
          <motion.div
            key={metric.key}
            custom={index}
            variants={ANALYTICS_CARD_VARIANTS}
            initial="hidden"
            animate="visible"
          >
            <BrutalCard
              className={metric.bg}
              title={metric.label}
              subtitle="Live planner data"
              cardTestId={`analytics-metric-${metric.key}-card`}
            >
              <p className="text-3xl font-semibold md:text-4xl" data-testid={`analytics-metric-${metric.key}-value`}>
                {analytics[metric.key]}
                {metric.suffix || ""}
              </p>
            </BrutalCard>
          </motion.div>
        ))}
      </div>

      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <BrutalCard className="bg-white" title="Completion Trend" subtitle="Task completion quality"
cardTestId="analytics-completion-card">
          <div className="space-y-4" data-testid="analytics-completion-wrap">
            <p className="text-sm md:text-base" data-testid="analytics-completion-rate-text">
              You are currently completing <strong>{completionRate}%</strong> of your tasks.
            </p>
            <div className="h-4 w-full border-2 border-[#18181b] bg-[#f4f4f5]" data-testid="analytics-completion-track">
              <div
                className="h-full bg-[#18181b] transition-[width] duration-300"
                style={{ width: `${completionRate}%` }}
                data-testid="analytics-completion-fill"
              />
            </div>
            <p className="text-sm" data-testid="analytics-upcoming-deadlines-text">
              Upcoming active deadlines: <strong>{analytics.deadlines_upcoming}</strong>
            </p>
          </div>
        </BrutalCard>

        <BrutalCard className="bg-[#bbf7d0]" title="Subject Distribution" subtitle="Where your effort is going"
cardTestId="analytics-subject-card">
          <div className="space-y-4" data-testid="analytics-subject-list">
            {analytics.subject_breakdown.length === 0 ? (
              <p className="text-sm" data-testid="analytics-subject-empty">No subject data yet. Add tasks to see
insights.</p>
            ) : (
              analytics.subject_breakdown.map((item, index) => {
                const ratio = item.total_tasks ? Math.round((item.completed_tasks / item.total_tasks) * 100) : 0;
                return (
                  <div key={`${item.subject}-${index}`} data-testid={`analytics-subject-item-${index}`}>
                    <div className="mb-1 flex items-center justify-between text-sm">
                      <span data-testid={`analytics-subject-item-${index}-name`}>{item.subject}</span>
                      <span data-testid={`analytics-subject-item-${index}-ratio`}>{ratio}% completed</span>
                    </div>
                    <div className="h-3 w-full border-2 border-[#18181b] bg-white"
data-testid={`analytics-subject-item-${index}-track`}>
                      <div
                        className="h-full bg-[#18181b] transition-[width] duration-300"
                        style={{ width: `${ratio}%` }}
                        data-testid={`analytics-subject-item-${index}-fill`}
                      />
                    </div>
                  </div>
                );
              })
            )}
          </div>
        </BrutalCard>
      </div>
    </section>
  );
}
```